

## Assertions

⇒ Assertions are primarily used to validate the behavior of a design → used in functional verification  
⇒ Assertions are used to verify correctness of a design by embedding checks directly into the design by ~~introduce~~ debugging time

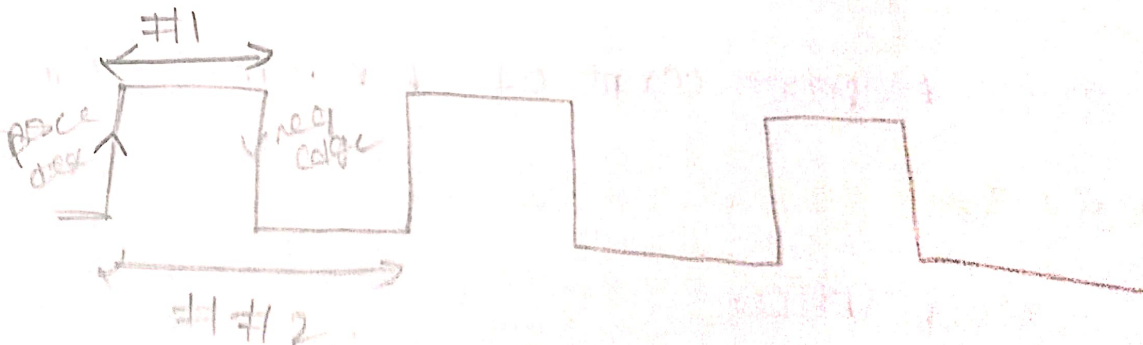
### TWO types of Assertions

- (i) immediate Assertions
- (ii) concurrent Assertions

### ~~(i) Immediate Assertions~~

(#1) → CLK delay → post edge CLK

(##1) → cycle delay → post edge neg edge CLK



Immediate Assertion = error will determine  
 Immediate Assertions check the condition  
 at a specific simulation time (instantly)

Syntax

label: assert (Expression) action block

pass \_statements, else fail \_statements

assert (Expression) & display("true"); else \$display("false");  
 assert (Expression) & display("true"); else \$warning("false");  
 assert (Expression) & display("true"); else \$error("false");  
 assert (Expression) & display("true"); else \$fatal("false");

always (posedge clk) assert (a & b);

$a \& b \rightarrow$  both are true  
 $a || b \rightarrow$  one is true  
 both are true

also assertion pass check on rise



concurrent Assertions  $\Rightarrow$  error will occur next cycle.

concurrent assertions check the scaliness of events spread over multiple clock cycles.

Syntax  
~~label: ~~property~~~~  
label: assert property (@(posedge clk) not(a <= b));

Create Boolean Expression

$\Rightarrow$  boolean expressions (arithmetic / logical operations)

Create Sequence Expression

$\Rightarrow$  group of expression will be a sequence

Create Property

$\Rightarrow$  group of sequences will be a property

assert property (↑)   
 trigger

Assert Property

$\Rightarrow$  property will be triggered & assert

sequence seq1

property (P);

@(posedge clk) (seq1 || seq2);

end property

end sequence

sequence seq2

a1: assert ~~property~~ (P);

0 <= 2 e;

$\downarrow$   
assert name

$\downarrow$   
property name



Implication operator  
 $\Rightarrow$  the implication is evaluation of if-then statement  
 $\Rightarrow$  the implication construct can be used only  
properties not in sequence

There are 2 types of Implication

- (1) Overlapped implication
- (2) non overlapped implication

(1) Over lapped

denoted " $a \rightarrow b$ "  $a \rightarrow b$  then "b" will be  
 $\downarrow$  agree

$\Rightarrow$  consider <sup>some</sup> that cycle

$\Rightarrow$  the consequent expression is evaluated in some

(2) non - overlapped

denoted " $a \Rightarrow b$ "

$\Rightarrow$  consider from next cycle.

$\Rightarrow$  the consequent expression is evaluated in the next

$a \rightarrow b$

$\downarrow$

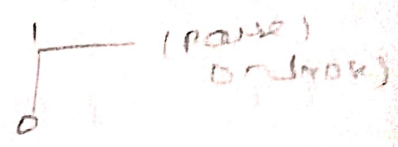
create another variable b contain

a fair and code exit smoothly

# SVA methods.

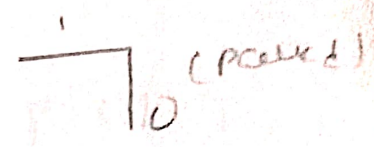
## \$rose

`$rose (boolean Expression (or) signal)`  
 ex: `$rose(a)`  
 0 to 1 → assertion pass



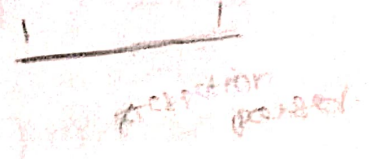
## \$fell

`$fell (boolean Expression (or) signal)`  
 ex: `$fell(a)` 1 to 0 → assertion pass.



## \$stable

`$stable (boolean Expression (or) signal)`  
 ex: `$stable(a)`



## \$past

`$past (signal name, no. of clock cycles);`

ex: `$past(a, 3)`

⇒ past before 3<sup>rd</sup> cycle a has to be high  
 don't assertion pass  
 → some value for clock  
 de past 1 cycle

